

C++2 Midterm Project

Phase I: Planning and Preparation

-Deliverable:

1. (50 points) c2m_summary.docx
 - a. **Summarize** the project to clarify the underlying issues.
 - b. Specify **input data type** and provide provisional **sample inputs & outputs**.

Phase II: Implementation and Documentation

-Deliverables:

2. (100 points) c2m_proj.cpp
Fully functional, error-free code implementing all features.
3. (50 points) c2m_proj.png
Screenshots showing the program's output, demonstrating each algorithm's functionality.
5. (100 points) YouTube link(s)
Video presentation of the algorithm and program execution.

Text-Based Battleship Game

Objective

Develop a console-based Battleship game that allows two players to compete against each other. The game should utilize classes to manage game elements and include functionality to save and load game progress.

Requirements

Classes

The following class specifications are the minimum requirements for the implementation of the Battleship game. Implementations are encouraged to add additional classes and/or methods as necessary to enhance functionality and improve code organization. All code must adhere to Object-Oriented Programming (OOP) principles.

Ship Class

Attributes:

- name: (string)
- size: (int)
- coordinates (vector of pairs) - stores the ship's position on the grid.
- hitPoints (int) - tracks how many hits the ship has taken.

Methods:

- Constructor to initialize the ship.
- placeShip(const vector<pair<int, int>>& coords): Places the ship on the grid.
- isSunk(): Returns true if the ship is sunk (hitPoints are zero).
- receiveHit(): Reduces hitPoints by one when hit.

Board Class

Attributes:

- grid (2D vector of characters) - represents the game board (e.g., 'O' for empty water, 'S' for ship, 'X' for hit).
- ships (vector of Ship objects) - stores all ships on the board.

Methods:

- initializeBoard(): Sets up the game board.
- placeShip(Ship& ship): Places a ship on the board.
- receiveAttack(int x, int y): Checks if an attack hits a ship and updates the board accordingly.
- displayBoard(): Displays the current state of the board.

Game Class

Attributes:

- player1Board (Board)
- player2Board (Board)
- currentPlayer (int) - tracks whose turn it is (1 or 2).

Methods:

- startGame(): Initializes the game and handles player turns.
- switchPlayer(): Changes the current player.
- saveGame(const string& filename): Saves the current game state to a file.
- loadGame(const string& filename): Loads game state from a file.

Game Flow

- Players take turns to attack the opponent's board by choosing coordinates (x, y).
- After each attack, the game should display the updated boards and indicate hits and misses.
- The game continues until one player has all their ships sunk.

Example of a Game Board (10x10)

Player 1's Board:

[illegible]

Player 2's Board:

[illegible]

In this example:

- 'O' represents empty water.
- 'S' represents a ship.
- 'X' represents a hit on a ship.

Note: If the game were real, we would not show the ship locations to the players. However, for the sake of debugging and evaluation, the ship locations are displayed here.

Players will use coordinates (x, y) to make their moves. For instance, if Player 1 attacks coordinates (3, 3), they will check Player 2's board to see if there is a ship at that location.

User Interface

Implement a main menu that provides the following options:

- Create a new game
- Load a previously saved game
- Exit the program

Once a game is started (new or loaded), present a gameplay menu during each player's turn that allows the user to:

- Make a move by entering coordinates (x, y) to attack.
- Save the current game progress at any time during their turn.

Provide clear instructions on how to input moves and save progress.

File Handling

- Use file handling to save the game state in a structured format (e.g., ship positions, player turns) to a text file (e.g., `battleship\_save.txt`).
- When loading a game, the program should restore the game state exactly as it was saved.

Documentation

- Include comments in your code to explain the purpose of each class and method.
- Include a video walk-through of your program code and execution

Evaluation Criteria

- Correctness: The program must meet all specified requirements and function correctly.
- Code Quality: Code should be clean, well-organized, and follow good programming practices.
- User Experience: The menu and user interactions should be intuitive and user-friendly.